

No Resource, No Benchmarks, No Problem? Evaluating and Improving LLMs for Code Generation in No-Resource Languages

Alessandro Giagnorio, Alberto Martin-Lopez, and Gabriele Bavota

Abstract—Large Language Models (LLMs) have significantly advanced the automation of software engineering tasks. One prominent example is code generation, where an LLM produces code in a specified programming language based on a natural language description. Most research in this area has focused on high-resource languages, such as Python or Java, which benefit from abundant training data in repository platforms like GitHub. A smaller body of work has explored low-resource languages (e.g., Lua, Racket), which are underrepresented in training corpora. In contrast, no-resource languages for which LLMs have seen virtually no training data remain largely unstudied. These languages often emerge in industry, where organizations develop proprietary or domain-specific languages unsupported by commercial tools like GitHub Copilot. This results in the need for companies to deploy their own in-house code recommenders. To investigate possible solutions in this context, we build and release three code generation benchmarks for no-resource languages, based on two recently proposed programming languages for which very little training data is available. Using these benchmarks, we experiment several solutions to teach LLMs about no-resource languages, including prompt-based techniques (e.g., few-shot) as well as pre-training and fine-tuning exploiting the little data available. While further pre-training gives the largest performance gains for no-resource languages, applying it directly to instruction-tuned models harms their ability to follow instructions. To address this, we start from a base model, further pre-training it on the target language, and then inject instruction-following capabilities via weight diff transfer from an instruction model. Such an approach significantly improves code generation capabilities in no-resource settings, allowing companies to cheaply deploy an instruct model specialized on the language of interest without dealing with the computational cost of instruction fine-tuning.

Index Terms—Code Generation, Benchmarks, Empirical Study, Large Language Models for Code

I. INTRODUCTION

Code generation is the task of automatically implementing source code from a higher-level specification, typically written in natural language. The majority of existing work on Large Language Model (LLM)-based code generation has focused on “high-resource programming languages,” such as Python and Java [1]–[3]. These languages are well represented in public code repositories, making them dominant in the pre-training corpora of LLMs. As a result, models tend to perform particularly well on these languages [4].

More recent studies have begun to investigate low-resource programming languages [4]–[6], such as Lua, R, and Racket,

which are characterized by relatively limited training data. While performance on these languages is generally lower than on high-resource ones, large-scale LLMs can still generalize reasonably well [6].

In contrast, little attention has been paid to no-resource programming languages, *i.e.*, languages that fall outside the pre-training distribution of LLMs. In particular, we focus on no-resource general-purpose languages, which share syntactic and semantic characteristics with mainstream programming languages but lack training data. As a consequence, commercial tools such as Copilot [7] or ChatGPT [8] do not support these languages, leaving organizations interested in AI-assisted programming with the challenge of developing custom, in-house solutions. Crafting effective and economically sustainable solutions is, however, far from trivial.

We make a number of contributions aimed at pushing forward research on code generation for *no-resource programming languages*. We start by building and releasing three code generation benchmarks for this context. Benchmarks are used to assess the code generation performance of LLMs, and consist of a collection of coding tasks, each providing a natural language description (or specification) and a test suite aimed at assessing whether the LLM correctly implements the code. To the best of our knowledge, there are no publicly available benchmarks for no-resource languages since, as said, those are usually proprietary languages. To overcome this problem, we take as representative of no-resource languages Gleam [9] and MoonBit [10], two languages recently released and unlikely to be relevant in the training data of LLMs. Indeed, only very few repositories written in these languages can be found on GitHub (280 Gleam and 35 MoonBit repositories with ≥ 10 stars), with their popularity being extremely lower than those of languages considered in previous work as low-resource [6] (e.g., 18k R and 19k Lua GitHub repositories with ≥ 10 stars). To build benchmarks for these languages, we translated three code generation benchmarks, namely HumanEval [11], MBPP [12], and the subset of “hard” coding problems featured in McEval [13]. Having the benchmarks for *no-resource languages*, we want to (i) confirm the expected lack of support provided by modern LLMs, and (ii) explore techniques which would allow companies to come up with a working solution at reasonable cost. We run four state-of-the-art LLMs (*i.e.*, GPT-4o [14], o3-mini [15], Qwen 2.5 Coder 32B Instruct [16], and Qwen 3 32B Instruct [17]) on our benchmarks, obtaining—as expected—a very low *pass@1* for both languages (*i.e.*, percentage of tasks for which the LLM was able to produce a

Alessandro Giagnorio and Gabriele Bavota are with SEART @ Software Institute, Università della Svizzera italiana. Alberto Martin-Lopez is with the SCORE Lab, I3US Institute, Universidad de Sevilla.

test-passing solution with 1 attempt). We also run the LLMs on the same three benchmarks but written in high-resource languages (Python and Java) and in languages considered by previous work as low-resource (Julia, Lua, R, Racket, Haskell). This was done to “set the bar” for what would be acceptable performance for a no-resource language. For example, if an LLM obtains $\sim 50\%$ *pass@1* for low-resource languages, it is reasonable to see the 50% as a sort of upper bound for its specialization to a no-resource language. To give an idea of the achieved results, we summarize the findings on the McEval-Hard benchmark, being the most challenging in our experiment: The four models achieve *pass@1* scores being in the range of ~ 59 - 89% for high-resource languages (depending on the LLM and the language), 27 - 84% for low-resource, and 0 - 1% for no-resource.

Then, we experiment on the same four LLMs techniques aimed at boosting their performance on the no-resource languages. These include two in-context learning approaches, *i.e.*, few-shot and Retrieval-Augmented Generation (RAG), as well as pre-training and fine-tuning on the little data that can be collected for the two languages. The further pre-training (on top of the one which was already performed by the LLM’s authors) was the best-performing technique, with the models approaching a $\sim 15\%$ *pass@1* on the no-resource languages (McEval-Hard). However, the further pre-training can only be applied on “base” LLMs, namely their non-instruct models. Indeed, further pre-training an instruct model can degrade their instruction-following capabilities [18], which is a key feature for an AI-based assistant. For this reason, we also experiment with an approach inspired by recent works in the Natural Language Processing (NLP) [18], [19] field that showed the possibility to transfer weights across LLMs having the same architecture. We thus start from a pre-trained base (non-instruct) model, M_b , for which an instruct version, M_i , is available. Then, we perform a further pre-training of M_b exploiting the data available for the no-resource language, since this was the most effective technique we experimented with. At this point, M_b acquires knowledge of the language k of interest ($M_b \rightarrow M_{bk}$). However, since it is not an “instruct” model, M_{bk} is not able to follow instructions. For this reason, we inject in it instruction-following capabilities by computing the weight diff between M_i and M_b and “add” such a diff to M_{bk} ’s weights. This results in a model that features all instruction-following capabilities of an instruct model without incurring in the computational cost of full instruction fine-tuning. This approach further boosts the capabilities of the experimented models on no-resource languages, with *pass@1* higher than 25% on the McEval-Hard benchmark.

In summary, our contributions are: (i) three benchmarks (HumanEval, MBPP, and McEval-Hard) translated in two no-resource languages (Gleam and MoonBit); (ii) an empirical study showing the gap in code generation performance LLMs experience when tested on high-, low-, and no-resource languages; and (iii) the experimentation of several techniques representing relatively cheap solutions allowing companies to deploy in-house coding assistants specialized for a language of interest.

II. STUDY DESIGN

The *goal* of the study is to experiment with techniques aimed at supporting the specialization of LLMs to no-resource languages. The *context* consists of nine languages, including high-, low-, and no-resource, and six LLMs, including commercial and open models. We aim at answering the following research questions (RQs):

RQ₁: *To what extent does the popularity of programming languages affect the code generation performance of LLMs?* This is a preliminary RQ, showing how the code generation performance of LLMs varies across languages characterized by a different amount of training data available on repositories such as GitHub. This will provide indications on (i) what are reasonable upper bounds expected for no-resource languages; and (ii) to what extent modern LLMs are able to cope with what have been considered in previous work as low-resource languages [4], [6]. In RQ₁, all LLMs are experimented in a zero-shot setting (*i.e.*, used out of the box).

RQ₂: *To what extent can in-context learning, pre-training, and fine-tuning boost the code generation performance of LLMs on no-resource languages?* In RQ₂ we assess whether few-shot, RAG, further pre-training and fine-tuning can help boosting the LLMs’ performance on no-resource languages. Few-shot consists in prompting the LLM with concrete examples of code generations in the no-resource language of interest before asking for a new code generation task. RAG, instead, injects in the prompt information taken from the languages’ documentation which is retrieved based on its relevance for the code generation task at hand. Concerning pre-training and fine-tuning, we use the little data available in GitHub repositories to teach the LLMs something about the no-resource languages.

A. Context Selection

In the following subsections we detail the context of our study in terms of (i) selected languages; (ii) code generation benchmarks; (iii) LLMs; and (iv) datasets used for pre-training and fine-tuning.

1) *Languages: No-Resource.* The main focus of our study is on two no-resource languages: Gleam and MoonBit. Gleam is a functional type-safe programming language designed for multi-thread scalability [9]. MoonBit is a general-purpose language designed for cloud and edge computing [10]. Both languages have been selected as representative of no-resource languages since their first stable versions have been proposed relatively recently, with Gleam v1 announced on 4 March 2024, and the MoonBit compiler made available on 18 December 2024. Note also that, being quite new, these languages are quickly evolving and, thus, even if recent LLMs may have seen some data about them during training, they may not have been exposed to the most recent languages’ features (*e.g.*, MoonBit introduced a “virtual packages” feature on 16 May 2025). Also, their popularity on GitHub is extremely low compared to other languages, making them almost irrelevant in the LLMs’ training sets.

Despite the existence of several no-resource languages, we focus only on Gleam and MoonBit for three reasons: (i) their stable release was launched after the cutoff date of

TABLE I
CONTEXT SELECTION: PROGRAMMING LANGUAGES.

Language	Classification	#GitHub Repos.
MoonBit	no-resource	400
Gleam	no-resource	2,900
Racket	low-resource	22,200
Julia	low-resource	81,000
Haskell	low-resource	155,000
Lua	low-resource	517,000
R	low-resource	981,000
Java	high-resource	18,700,000
Python	high-resource	21,500,000

the evaluated LLMs; (ii) they are documented well enough for the authors to gain proper expertise for the creation of the benchmarks; and (iii) they provide community support, to ask questions in case of doubts. These criteria are important to ensure that the benchmarks we create for no-resource languages are of high quality and that the evaluated LLMs have likely not seen significant training data about them.

To give an idea of the different amounts of data available for the nine languages considered in our study, Table I shows their number of public GitHub repositories as collected on 2 July 2025. The classification as low- or high-resource languages follows previous studies in the literature [4]–[6], [20], while the reported number of repositories per language helps in clarifying why Gleam and MoonBit can be considered as “no-resource”. Indeed, they have at least one order of magnitude less GitHub repositories as compared to the least popular low-resource language (Racket). It is also important to consider that before March 2024 only 560 Gleam and 7 MoonBit GitHub repositories existed. This is a relevant date since, as we will explain later, it is the latest cutoff date for four of the LLMs considered in our study (*i.e.*, the LLM having the most recent training data has seen data up to March 2024). No official cutoff date is available for the remaining LLMs.

Low-Resource. As done in recent studies [4]–[6], [20], we consider Racket, Julia, Haskell, Lua, and R as low-resource languages. As visible from Table I, the amount of potential training data they offer is substantially lower than that of high-resource languages. Still, within the set of low-resource languages we consider, there is a strong variability in their popularity, going from the $\sim 22k$ repositories of Racket up to the $\sim 981k$ of R. We study the extent to which their popularity impacts the LLMs’ performance in RQ₁.

High-Resource. Java and Python are representative of high-resource languages in our study, with $>10M$ repositories each.

2) *Benchmarks:* To evaluate the code generation capabilities of the LLMs on the nine languages we resort to three benchmarks, namely HumanEval, MBPP, and McEval-Hard, with the last being a novel benchmark we propose. All these benchmarks exercise LLMs in function-level code generation (*i.e.*, given the description and signature of a function, finalize the implementation). We acknowledge that more complex and realistic benchmarks exist, like those tasking the LLMs with implementing the changes described in real issues (see *e.g.*,

SWE-Bench [21]). However, we decided to keep our focus on function-level benchmarks for mainly two reasons. First, our target on no-resource languages questions the ability of LLMs to cope with even self-contained and focused implementation tasks. Second, adopting more complex benchmarks would hinder a fair comparison across high-, low-, and no-resource languages, as it would require collecting semantically equivalent issues for multiple programming languages, a requirement that is difficult to satisfy in practice. In the following, we describe the three benchmarks.

HumanEval and MBPP. The first two have been taken from the MultiPL-E benchmark proposed by Cassano *et al.* [5]. MultiPL-E features the coding tasks of these two benchmarks translated to 24 programming languages, including all the high- and low-resource languages considered in our study. Each coding task represents a specific function to implement, described in natural language, and having associated tests to evaluate the correctness of the LLM’s implementation.

To assess the performance of the LLMs on the no-resource languages, we also translated these two benchmarks in Gleam and MoonBit. The translation was performed by the first two authors starting from the Python version of the benchmarks and following a translation pipeline we defined. First, as done by Cassano *et al.* [5], we had to exclude 6 out of 161 tasks from HumanEval and 27 out of 399 in MBPP since they were too Python-specific and could not be translated into other languages. Also, a few problems were not available in MultiPL-E in all high- and low-resource languages subject of our study. Thus, to have a fair comparison among all languages, we considered only those available for all languages, which led to a final number of 154 problems for HumanEval and 355 for MBPP. For these coding tasks, we started by translating their prompts. Indeed, not only the function’s signature is obviously different for each language, but also the textual description (docstring) requires translation. The problem arises from the fact that different languages use different terms to refer to the same coding construct. For example, an `Array` in Python corresponds to a `FixedArray` in MoonBit, while a Python’s `List` corresponds to an `Array` in MoonBit. To have a starting point for the prompts translation, we instructed ChatGPT to implement these changes.

This was done by explicitly providing ChatGPT with (i) all textual transformations we expected (*e.g.*, `Array` $\rightarrow$ `FixedArray`) and (ii) examples of good translations. The generated prompt translations were automatically checked for syntax errors in the signature and, then, manually inspected to find errors both in the signature and description. After obtaining the translated prompts, we proceeded to the translation of test suites. Once again, we started from translations proposed by ChatGPT: we provided the LLM with (i) the original coding task, featuring both the original prompt and test suite; (ii) the translated prompt manually checked; and (iii) examples of good translations. The generated tests again represent a starting point and went through an automated syntax check and a subsequent manual inspection. The tests required major fixes due to ChatGPT’s lack of knowledge of the two languages.

McEval-Hard. The third benchmark considered in our study is McEval-Hard, that we built starting from McEval [13].

TABLE II
CONTEXT SELECTION: LLMs.

	# Trainable # Parameters	Instruction Following	Reasoning Capabilities	Cutoff Date
Qwen 2.5 Coder Base [22]	32B	✗	✗	2024-03
Qwen 2.5 Coder Instruct [16]	32B	✓	✗	2024-03
Qwen 3 Base [23]	8B	✗	✗	?
Qwen 3 Instruct [17]	32B	✓	✓	?
o3-mini [15]	~200B	✓	✓	2023-10
GPT-4o [14]	~200B	✓	✗	2023-10

McEval features coding tasks in 40 programming languages, including all the high- and low-resource ones considered in our study (but none of the no-resource). The coding tasks within each language are organized in three categories, based on their difficulty: easy, middle, hard. However, the coding tasks are not the same for the 40 languages, making a comparison of the LLMs' performance across the languages difficult. Also, there are only a few tasks per language (~50), out of which <10 are hard problems. To build a challenging code generation benchmark being equal for all languages, we collected the "hard tasks" from McEval in each of the 40 languages (385 in total), filtered those without any tests (35 tasks), removed the 87 duplicated ones (*i.e.*, those present in more than one language) and those being too specific of the source language. This left us with 227 tasks that we translated in the nine languages subject of our study. The adopted translation pipeline is the same previously described for HumanEval and MBPP.

Knowing that translating benchmarks can be error-prone, we also looked for the possibility of having the benchmarks for the no-resource languages double-checked by the creators of the languages themselves. We managed to have such a double check at least for Gleam (<https://lpil.uk/>) for a random sample of 50 instances: The feedback provided did not spot any major issue in our translation, but mostly recommended stylistic improvements which did not alter the code behavior, but made the code more compliant to the Gleam syntax.

3) *LLMs*: We selected six LLMs diversifying between open and commercial, and with/without instruction-following and/or reasoning capabilities. Table II lists the selected LLMs, specifying their size, instruction-following and reasoning capabilities, and cutoff date, namely the date in which the collection of their training data has stopped. The latter information is not publicly available for all models (see question marks in the table). For the OpenAI models (*i.e.*, o3-mini and GPT-4o), the reported number of parameters is estimated, since such information is not publicly available.

In RQ₁ the four LLMs with instruction-following capabilities are experimented in zero-shot on all nine programming languages: Qwen 2.5 Coder 32B Instruct, Qwen 3 32B Instruct, o3-mini, and GPT-4o. In RQ₂, instead, we consider all of them, since different LLMs are suitable to experiment with different strategies to boost their performance on no-resource languages. In particular:

In-context learning techniques (*i.e.*, *few-shot* and *RAG*) are experimented on models having instruction-following capabilities: Qwen 2.5 Coder 32B Instruct, Qwen 3 32B Instruct, o3-mini, and GPT-4o.

Pre-training is experimented on non-instruct models, since further pre-training on instruct models is known to result in catastrophic forgetting of the instruction capabilities [18]: Qwen 2.5 Coder 32B Base and Qwen 3 8B Base. Note that for Qwen 3 a 32B Base (*i.e.*, non Instruct) model is not available, thus explaining our choice of the 8B parameters here.

Fine-tuning is experimented on open instruct models, since it cannot be performed on closed models: Qwen 2.5 Coder 32B Instruct and Qwen 3 32B Instruct.

4) *Datasets Used for Further Pre-Training and Fine-Tuning on Gleam and MoonBit*: We collected Gleam and MoonBit code from public GitHub repositories. Since these languages are both very recent, we defined a cut-off date to extract only up-to-date code: For Gleam we mined only repositories created after 5 March 2024 (*i.e.*, the day after the first stable version has been released). For MoonBit, instead, since we do not have an official date for the first release, we simply collected from all its repositories all files created in 2025, to ensure we capture the latest grammar and language features. This resulted in code files coming from a total of 2,159 Gleam and 262 MoonBit repositories.

Pre-Training. To avoid data leakage between the collected Gleam and MoonBit files and the used benchmarks, we perform a two-step process. First, we automatically remove all files containing a function having the same name of one of the functions in the benchmarks. Second, as done in recent work [24], [25], we extract all 8-grams composing each coding task featured in our benchmarks and check whether it is found in any of the pre-training files. In case of a match, the first author checked whether this was an actual case of data leakage or not, excluding the file from the pre-training dataset in the first case. At the end of this process, we obtained 18,767 Gleam and 3,609 MoonBit files for pre-training. In addition, we crawled the official documentation of the two languages from their respective websites. This documentation includes an overview of the languages, course materials, language cheat sheets, and descriptions of the standard libraries. The documentation is also part of the pre-training dataset.

Fine-Tuning. Starting from the code files in the pre-training dataset, we extracted all functions using the `tree-sitter`¹ library. The `tree-sitter` parsers are developed by the very same Gleam² and MoonBit³ developers, thus we expect them to be reliable. Based on the functions extracted, we filtered out those not having a docstring, with non-ASCII characters, or with a description shorter than 10 characters. Finally, we removed all functions having an empty body, TODO implementations, or being duplicated (*e.g.*, the same function is present across different pre-training files). Also, we performed the same two-step data leakage checks mentioned for the pre-training datasets, removing all suspect functions. This resulted in 13,534 Gleam and 2,444 MoonBit functions for the fine-tuning datasets. Table III reports the total number of tokens in the pre-training and fine-tuning datasets, including

¹<https://github.com/tree-sitter/tree-sitter>

²<https://github.com/gleam-lang/tree-sitter-gleam>

³<https://github.com/moonbitlang/tree-sitter-moonbit>

TABLE III
NUMBER OF TOKENS IN GLEAM AND MOONBIT DATASETS.

Dataset	Language	# Code Tokens	# Doc Tokens	# Total Tokens
Pre-training	Gleam	28.2M	0.1M	28.3M
	MoonBit	13.1M	0.6M	13.7M
Fine-tuning	Gleam	3.6M	—	3.6M
	MoonBit	0.5M	—	0.5M

both code and documentation tokens (computation done using the Qwen2.5 Coder tokenizer).

B. Data Collection

1) *RQ₁*: In *RQ₁* the LLMs are run in zero-shot on the three benchmarks of each of the nine languages using the prompts available in our replication package [26]. All prompts feature for each coding task the natural language description and the signature of the function to implement. We set the temperature to 0.2 when generating predictions for all models but o3-mini, which does not allow any temperature setting (thus, the default is used in this case). The temperature is used to control the randomness of the model's predictions, with 0 being the lowest and 2 the highest. The 0.2 setting is aligned with previous work in the literature (see *e.g.*, [5], [6], [11]).

To account for the stochastic nature of LLMs, each model was run 10 times on each benchmark and language. In total, we performed 66,240 code generations with each model in zero-shot, for a total of 264,960 generations ($66,240 \times 4$ models). All generations have been run against the respective test suites, to assess their correctness.

2) *RQ₂*: We experiment with four strategies aimed at boosting LLMs' performance on the no-resource languages: *few-shot*, *retrieval-augmented generation* (RAG), *pre-training*, and *fine-tuning*. At inference time, we apply the same parameters used in *RQ₁*.

Few-Shot. We use the fine-tuning dataset described in Section II-A4 as the *knowledge base* from which to retrieve code examples. These examples are first transformed into embeddings using OpenAI's `text-embedding-3-large` model [27], and then indexed using the FAISS library [28]. During code generation, we transform the benchmark prompt into embeddings and retrieve the top-five most similar examples from the processed *knowledge base*. These examples are prepended to the final code generation prompt.

RAG. We use the official documentation of the two no-resource languages as source for retrieval-augmented generation. Similarly to *few-shot*, we transform the documentation into embeddings using the `text-embedding-3-large` model and store them in a vector database. Each vector is a "subset" of the language documentation (a paragraph) represented as an embedding.

To retrieve the most relevant documentation for a given coding task, we follow a multi-step process guided by an LLM M_e , which in our implementation was `gpt-4o-mini-2024-07-18` [29]:

- 1) **Planning**: we prompt M_e to generate a language-agnostic step-by-step plan for the coding task.

- 2) **Query Generation**: for each step of the plan, we ask M_e to generate a query that can be used to retrieve relevant documentation snippets from the vector database. For example, for a step like "sort the list in input", the query could be "*How to sort a list?*".
- 3) **Retrieval**: for each query, we retrieve the five most relevant portions of documentation from the vector database and summarize them using M_e .

Finally, we concatenate the generated queries and their summaries and use them to augment the task context.

Pre-Training. We further pre-train the base versions of Qwen 2.5 Coder 32B and Qwen 3 8B on the pre-training datasets previously described. These datasets are chunked into sequences of 2,048 tokens, which we set as the maximum sequence length during training. Models are pre-trained using the Causal Language Modeling (CLM) objective and the *LoRA* [30] technique. We adopt the same *LoRA* hyperparameters used in [31], which are $r = 16$, $\alpha = 32$, and $dropout = 0.05$. The pre-training is performed for five epochs, using a learning rate of 5×10^{-5} , AdamW optimizer [32], and a linear scheduler having a decaying factor of 0.

Fine-Tuning. Similarly, we fine-tune the chosen LLMs with a maximum sequence length of 4,096 tokens on the fine-tuning datasets mentioned above. We use *LoRA* with the same hyperparameters as for pre-training, and we train our models for five epochs using the recommended parameters from vendors, *i.e.*, learning rate of 5×10^{-5} , AdamW optimizer, and a cosine scheduler. For inference, we select the last epoch of each model, as the training loss converged in this epoch (see replication package [26]).

C. Data Analysis

The reference metric for evaluation and comparison is *pass@1*, where 1 indicates the number of "attempts" a model is allowed to make. If the model's code passes *all* unit tests for a given task, then *pass@1* = 1; otherwise, *pass@1* = 0. In addition, to capture partial correctness, we also report the percentage of passed unit tests (*passed%*), which provides a more fine-grained view of model performance in cases where not all tests are satisfied. For example, given a coding task having four tests and a candidate implementation provided by an LLM, we may have *pass@1* = 0 and *passed%* = 0.75 in the case in which the implementation results in three out of four tests passing. For the purpose of statistical significance, we run each LLM 10 times on each coding task in all RQs and compute both metrics with $n = 10$ repetitions.

In *RQ₁*, we use such metrics to compare the performance of the four LLMs across the nine languages, to observe the relationship between language popularity and LLMs' performance. In *RQ₂* the focus is on the no-resource languages, and in particular on how *few-shot*, RAG, *pre-training*, and *fine-tuning* can boost performance as compared to a zero-shot setting. Besides showing the performance achieved by LLMs with the different strategies (zero-shot, *few-shot*, RAG, *pre-training*, *fine-tuning*), we also statistically compare these strategies against zero-shot, to assess whether the provided boost is significant. To make a concrete example, when contrasting the performance of o3-mini in zero-shot vs *few-shot*

on HumanEval-Gleam in terms of $pass@1$, we consider two distributions composed of 154 coding tasks $\times$ 10 repetitions = 1,540 $pass@1$ values. We use the McNemar’s test [33], which is suitable to do pairwise comparisons of dichotomous results of two different treatments.

We adjust p -values using the Benjamini-Hochberg procedure [34] to account for multiple comparisons (*e.g.*, the performance of o3-mini in zero-shot is compared against what the same model achieves using few-shot, and RAG). We complement the McNemar’s test with the Odds Ratio (OR) effect size to quantify the magnitude of the differences between the experimented methodologies.

D. Replication Package

We provide in our replication package [26]:

- The *benchmarks* used in the form of JSONL files.
- The model *generations* collected across all RQs.
- The *prompts* used in the experiments.
- The *scripts* to replicate our experiments, from the data collection down to the implementation of all experimented techniques.
- Additional *results* discussed in the paper but not fully reported for the sake of brevity.

III. RESULTS DISCUSSION

We discuss our findings by research question. Since our results are consistent between $pass@1$ and $passed\%$, we only discuss the $pass@1$ findings, providing all data about $passed\%$ in our replication package [26].

RQ₁: Effects of Language Popularity on LLMs’ Code Generation Performance: Table IV illustrates the performance of the four selected LLMs when used in a zero-shot setting (*i.e.*, out of the box) on high- (Python, Java), low- (R, Lua, Haskell, Julia, Racket), and no-resource (Gleam, MoonBit) programming languages, across all three benchmarks we experimented with. We discuss the results along two dimensions, namely language families and benchmarks. Languages are sorted from left to right based on their popularity on GitHub, as shown in Table I.

All studied LLMs exhibit high performance when evaluated on high-resource programming languages. The $pass@1$ obtained for these languages ranges between 59% (Java, McEval-Hard, GPT-4o) and 97% (Python, HumanEval, o3-mini), with an average performance of 79% across all models and benchmarks.

Results on low-resource languages, while more conservative, show in several cases performance comparable to those of high-resource languages. The $pass@1$ scores range between 27% (Haskell, McEval-Hard, Qwen 3) and 87% (Haskell, HumanEval, o3-mini), with an average of 62%. In 49 out of 60 cases (5 languages $\times$ 3 benchmarks $\times$ 4 models), $pass@1$ is above 50%. Even open-source models achieve over 50% $pass@1$ in 20 out of 30 cases, indicating that low-resource languages may not be as challenging anymore as previously reported in the literature [4]–[6] thanks to progress in LLMs’ capabilities. Moreover, the distribution of results in Table IV demonstrates that the performance of low-resource

languages is not solely determined by their popularity. For instance, LLMs consistently perform better on Lua than on R, although the former has a smaller number of GitHub repositories compared to the latter (see Table I). As already observed in previous studies [6], [35], it is possible that the similarity with a high-resource language can help the model perform well on a low-resource language. For example, in this specific case, Lua is more similar to Python than R.

Results on no-resource languages are, as expected, radically different. Performance across LLMs and benchmarks stays between 0% and 20%, with an average of 9%. The $pass@1$ scores $\geq 10\%$ achieved for Gleam and MoonBit on HumanEval and MBPP may appear surprising, but they can be explained by the fact that these benchmarks feature trivial coding tasks such as “provide the sum of two integer numbers” or “write a function to find the volume of a cube given its side length”.

Listing 1. Trivial task from MBPP benchmark for MoonBit.

```
1 // Write a function to find the volume of a
2 // cube given its side length.
3 fn volume_cube(l: Int) -> Int {
4     return l * l * l;
5 }
```

The latter case is depicted in Listing 1 for the MoonBit language and shows that such a coding task is not only trivial, but can be solved with knowledge of similar high-resource languages. Indeed, it is worth remembering that the LLM’s prompt includes the function signature (*i.e.*, `fn volume_cube(l: Int) -> Int {}`) which, thus, the LLM does not need to generate. The body “`return l * l * l;`” is common to many high-resource languages, such as Java. We also notice that the function’s signature in MoonBit is quite similar to Rust ($\sim 1\text{M}$ repositories on GitHub) that, as Java, supports the “`return l * l * l;`” function implementation. Thus, LLMs may leverage their knowledge of similar languages to successfully generate completions for trivial coding tasks in no-resource languages. As it can be seen from Table IV, as soon as the complexity of the coding task increases (our McEval-Hard benchmark), all LLMs consistently fail in no-resource languages (best $pass@1$ is 1% by o3-mini on MoonBit).

We also performed an analysis of the reasons behind the LLMs’ wrong code generations. In particular, we classified each wrong code generation as due to *syntactic* or *semantic* errors. The former identify code generations that violate the formal grammar of the target language (*i.e.*, there exists no parse tree / AST for it under that grammar). The latter, instead, are code generations that, while syntactically correct, result in either a runtime error or in at least a failing test. For the sake of brevity, full results are reported in our replication package [26], while here we discuss the main findings.

We found a clear trend that differentiates no-resource languages from the high- and low-resource ones. Indeed, in Gleam and MoonBit the vast majority of failures are due to syntactic errors, indicating that LLMs struggle with the basic syntax of these languages. For instance, for the best-performing model on these languages (*i.e.*, GPT-4o), roughly two-thirds of the failures are syntactic. This proportion is even higher for other models (*e.g.*, up to 90% for o3-mini on

TABLE IV
LLMs PERFORMANCE ($pass@1$) ON HIGH-, LOW-, AND NO-RESOURCE PROGRAMMING LANGUAGES.

LLMs comparison on different programming languages									
GPT-4o									
Dataset	Python	Java	R	Lua	Haskell	Julia	Racket	Gleam	MoonBit
HumanEval	91.23	83.96	64.35	81.69	64.03	72.40	60.00	7.60	12.60
MBPP	84.90	76.11	64.85	70.85	66.62	74.00	63.66	20.31	16.54
McEval-Hard	77.67	59.30	55.02	65.42	53.79	52.42	32.25	0.97	0.88
o3-mini									
Dataset	Python	Java	R	Lua	Haskell	Julia	Racket	Gleam	MoonBit
HumanEval	96.75	92.92	74.48	84.03	87.21	83.70	79.68	4.55	7.34
MBPP	84.34	73.41	66.87	67.44	75.55	77.92	66.56	7.75	15.01
McEval-Hard	88.94	86.39	80.13	69.12	83.96	73.17	52.07	0.18	1.10
Qwen 2.5 Coder 32B Instruct									
Dataset	Python	Java	R	Lua	Haskell	Julia	Racket	Gleam	MoonBit
HumanEval	89.61	83.31	56.49	76.04	49.48	69.74	68.12	6.49	11.04
MBPP	83.32	66.82	62.56	70.14	51.01	74.68	57.27	9.83	17.80
McEval-Hard	70.57	72.60	47.89	62.07	31.63	48.41	33.00	0.40	0.88
Qwen 3 32B Instruct									
Dataset	Python	Java	R	Lua	Haskell	Julia	Racket	Gleam	MoonBit
HumanEval	89.03	77.66	55.26	80.52	46.82	70.84	61.88	4.55	7.27
MBPP	83.66	71.27	62.08	67.30	52.90	75.01	53.92	6.23	18.06
McEval-Hard	70.04	64.71	40.26	54.54	27.00	44.80	28.19	0.44	0.88

Gleam). Differently, in the high- and low-resource languages, syntactic errors represent a minority of the failures (typically below 10%), suggesting that LLMs generally possess a solid knowledge of their syntax. A notable exception is Java, which consistently exhibits a higher fraction of syntactic failures ($\sim 30\%$). A plausible explanation for such a finding is Java's syntactic verbosity, which requires the correct placement of multiple mandatory elements (e.g., class declarations, method signatures, types, and modifiers). Nevertheless, the overall success rate (i.e., $pass@1$) for Java is substantially higher than for no-resource languages. Consequently, although the relative proportion of syntactic failures appears high for Java, the absolute number of syntactic errors is considerably smaller than for Gleam and MoonBit.

Answer to RQ₁

LLMs show excellent code generation performance on high-resource programming languages, with $pass@1$ scores close to 100% in some cases. Low-resource languages show reasonable support with $pass@1$ scores above 50% in most cases, with popularity not being the only determining factor. No-resource languages remain a challenge, with $pass@1$ scores below 20% in most cases, and close to 0% for hard coding tasks (McEval-Hard). Such a low performance is mainly due to the inability to produce syntactically-correct code for these languages, which leads to failures even for the most trivial tasks.

RQ₂: Boosting LLMs' Performance on No-Resource Languages via In-Context Learning, Pre-Training, and Fine-Tuning: We investigate techniques aimed at improving LLMs' performance on Gleam and MoonBit.

Table V presents the $pass@1$ on the three benchmarks of the six LLMs subject of this RQ using zero-shot (as in RQ₁) plus four new strategies: *5-shot*, *RAG*, *fine-tuning*, and *pre-training*. As explained in Section II-A3, not all techniques have been experimented on all LLMs: *few-shot* and *RAG* are experimented on all models having instruction-following

capabilities (GPT-4o, o3-mini, Qwen 2.5 Coder 32B Instruct, and Qwen 3 32B Instruct); *fine-tuning* on open instruct models (Qwen 2.5 Coder 32B Instruct and Qwen 3 32B Instruct); and *pre-training* on non-instruct models only (Qwen 2.5 Coder 32B Base and Qwen 3 8B Base).

Each $pass@1$ value in Table V is accompanied by the Odds Ratio (OR) output of the McNemar's test comparing the performance of an LLM_i with a given technique (e.g., few-shot) versus the performance of LLM_i used in zero-shot (baseline). For example, applying few-shot to GPT-4o on Gleam, increases the $pass@1$ on HumanEval from 7.60% to 15.45%, resulting in an OR=8.56, which indicates that, among the discordant cases, few-shot was over 8 times more likely than zero-shot to produce a correct implementation. The few ORs that are not statistically significant are reported in grey in Table V. Note that a higher OR does not always imply a higher gap in $pass@1$. Indeed, McNemar's OR quantifies the imbalance in discordant outcomes (i.e., coding tasks for which one technique produces a correct output and the other does not). For example, a higher OR may arise from a small number of discordant cases with a strong directional imbalance, while a lower OR might come from a larger number of discordant cases that are more balanced.

Starting from in-context learning techniques (i.e., 5-shot and RAG), we observe that few-shot is slightly more effective than RAG. Indeed, in 7 out of 12 cases for Gleam and 8 out of 12 for MoonBit, few-shot outperforms RAG in terms of $pass@1$. We hypothesize that models are better able at grasping the grammar of unfamiliar languages from code examples rather than from relevant portions of the documentation, which may be more or less code-oriented depending on the language. This is partially confirmed by the fact that few-shot reduces syntax errors by 15.36% compared to zero-shot, while RAG achieves a smaller reduction of 8.94%.

It can also be seen from Table V that the boost in performance provided by in-context learning techniques is benchmark- and language-dependent. Indeed, such an improvement is higher (i) on MoonBit than on Gleam; and (ii) on

TABLE V
LLMs PERFORMANCE ON GLEAM AND MOONBIT WHEN USING ZERO-SHOT, FEW-SHOT, RAG, PRE-TRAINING, AND FINE-TUNING.

Model performance with different settings on no-resource programming languages										
GPT-4o										
Gleam						MoonBit				
Dataset	0-shot	5-shot	RAG	Fine-tuned	Pre-trained	0-shot	5-shot	RAG	Fine-tuned	Pre-trained
HumanEval	7.60	15.45 (8.56)	14.22 (21.40)	—	—	12.60	25.97 (5.29)	22.86 (3.59)	—	—
MBPP	20.31	30.37 (3.19)	24.03 (1.54)	—	—	16.54	40.51 (10.35)	32.68 (4.43)	—	—
McEval-Hard	0.97	1.23 (1.32)	2.16 (3.45)	—	—	0.88	8.06 (17.30)	6.12 (12.90)	—	—
o3-mini										
Gleam						MoonBit				
Dataset	0-shot	5-shot	RAG	Fine-tuned	Pre-trained	0-shot	5-shot	RAG	Fine-tuned	Pre-trained
HumanEval	4.55	9.74 (17.00)	9.29 (8.30)	—	—	7.34	39.22 (31.69)	32.08 (10.07)	—	—
MBPP	7.75	19.66 (6.35)	18.87 (5.76)	—	—	15.01	45.92 (23.85)	40.79 (8.32)	—	—
McEval-Hard	0.18	0.93 (6.67)	0.57 (5.50)	—	—	1.10	12.20 (19.00)	9.65 (13.12)	—	—
Qwen 2.5 Coder 32B Instruct										
Gleam						MoonBit				
Dataset	0-shot	5-shot	RAG	Fine-tuned	Pre-trained	0-shot	5-shot	RAG	Fine-tuned	Pre-trained
HumanEval	6.49	7.79 (3.00)	14.29 (13.00)	24.74 (32.22)	—	11.04	23.25 (9.17)	27.08 (4.48)	34.74 (11.43)	—
MBPP	9.83	18.87 (4.21)	21.44 (5.58)	34.03 (14.85)	—	17.80	33.86 (4.08)	30.48 (3.03)	37.38 (5.34)	—
McEval-Hard	0.40	1.32 (3.33)	0.88 (23.00)	3.04 (121.0)	—	0.88	5.81 (57.00)	7.05 (15.00)	10.93 (457.0)	—
Qwen 3 32B Instruct										
Gleam						MoonBit				
Dataset	0-shot	5-shot	RAG	Fine-tuned	Pre-trained	0-shot	5-shot	RAG	Fine-tuned	Pre-trained
HumanEval	4.55	9.09 (141.0)	10.39 (181.0)	23.57 (30.30)	—	7.27	22.40 (7.30)	23.96 (6.98)	34.81 (13.85)	—
MBPP	6.23	23.94 (16.34)	19.46 (11.22)	37.32 (37.80)	—	18.06	32.59 (6.21)	32.48 (3.81)	41.94 (12.94)	—
McEval-Hard	0.44	0.88 (2.00)	1.32 (3.00)	3.88 (8.80)	—	0.88	5.46 (11.40)	6.48 (255.0)	13.04 (553.0)	—
Qwen 2.5 Coder 32B Base										
Gleam						MoonBit				
Dataset	0-shot	5-shot	RAG	Fine-tuned	Pre-trained	0-shot	5-shot	RAG	Fine-tuned	Pre-trained
HumanEval	1.95	—	—	—	32.99 (957.0)	7.14	—	—	—	41.62 (18.70)
MBPP	3.46	—	—	—	47.35 (120.8)	12.42	—	—	—	44.76 (9.63)
McEval-Hard	0.00	—	—	—	12.47*	1.67	—	—	—	25.86 (35.31)
Qwen 3 8B Base										
Gleam						MoonBit				
Dataset	0-shot	5-shot	RAG	Fine-tuned	Pre-trained	0-shot	5-shot	RAG	Fine-tuned	Pre-trained
HumanEval	1.62	—	—	—	18.57 (19.64)	8.18	—	—	—	36.82 (18.64)
MBPP	4.65	—	—	—	23.63 (22.06)	12.79	—	—	—	42.08 (10.12)
McEval-Hard	0.35	—	—	—	4.36 (12.38)	0.44	—	—	—	19.87 (50.00)

*OR not computable since the LLM in 0-shot achieves 0.00 *pass*@1.

simpler coding tasks (HumanEval and MBPP) than on more complex ones (McEval-Hard). The higher gain on MoonBit than on Gleam can be explained by two factors. First, as shown in Table I, MoonBit has seven times less GitHub repositories than Gleam. Thus, for this language, we can conjecture that the additional information about the language provided via in-context learning is likely to make a stronger difference. Second, as stated in the paper presenting MoonBit [10], this language has been designed to be AI-friendly, with an AI-driven language design (see Section 2 in [10]) also featuring aspects of the language allowing a “*more flexible retrieval-based prompt augmentation*” [10].

As per the coding task complexity, it can be seen that on McEval-Hard the gain in performance is substantially lower for both languages, suggesting that showing examples (few-shot) or relevant parts of the documentation (RAG) in the prompt is not enough when dealing with challenging programming tasks.

Focusing on training-based approaches, the fine-tuned Qwen 2.5 Coder 32B Instruct and Qwen 3 32B Instruct outperform zero-shot and in-context learning techniques applied on the same models. Notably, fine-tuned open-source models often outperform commercial models in their best setting, especially on Gleam. For example, the Gleam fine-tuned version of Qwen 3 32B Instruct achieved 23.57% on HumanEval, 37.32% on

MBPP, and 3.88% on McEval-Hard, which can be compared against the best LLM with in-context learning (*i.e.*, GPT-4o with 5-shot) which achieved on the same three benchmarks 15.45%, 30.37%, and 1.23%, respectively. On MoonBit, instead, the best LLM using in-context learning is o3-mini which performs slightly better than the fine-tuned Qwen 3 32B Instruct (see Table V). In summary, given the same models, fine-tuning is superior to in-context learning. Also, fine-tuned open models are competitive with commercial ones used with few-shot or RAG.

The last technique we analyze is the further pre-training of the base models. Let us start from the results achieved on Qwen 2.5 Coder 32B Base, which can be compared against what we observed for its “instruct” version. For the latter, fine-tuning was by far the best-performing technique. Thus, we use it as comparison against its pre-trained base version. On all benchmarks and for both languages, the pre-trained base model is superior to the fine-tuned instruct model. Remember that we are comparing two models having the same size and the same architecture, with the instruct version just being a further instruction-tuned version of the base model. The gap in performance is major in all cases. For Gleam: on HumanEval, 32.99 (pre-trained) vs 24.74 (fine-tuned); on MBPP, 47.35 vs 34.03; and on McEval-Hard, 12.47 vs 3.04. For MoonBit: on HumanEval, 41.62 vs 34.74; on MBPP, 44.76 vs 37.38; and

on McEval-Hard, 25.86 *vs* 10.93. All these differences are statistically significant (McNemar test, p -values < 0.05 after Benjamini-Hochberg correction), with ORs ranging from 1.89 to 4.24 for Gleam and from 1.43 to 3.78 for MoonBit.

When looking at the pre-trained Qwen 3 8B Base, in this case we do not have an identical model to compare with. However, when looking at the results, we can see that on MoonBit the pre-trained 8B model has comparable performance to the fine-tuned 32B model: on HumanEval, 36.82 (pre-trained) *vs* 34.81 (fine-tuned); on MBPP, 42.08 *vs* 41.94; and on McEval-Hard, 19.87 *vs* 13.04. On Gleam, instead, the larger fine-tuned model is superior on HumanEval and MBPP, while worst on McEval-Hard. These differences are statistically significant in 1 case in favor of the 8B pre-trained model, and in 2 cases of the 32B fine-tuned model, not showing a clear winner.

Putting all above-discussed evidence together, we conclude that a further pre-training helps more than fine-tuning for no-resource languages. This can be explained by the different amount of data that can be exploited in the two training scenarios, as visible from Table III. Indeed, when pre-training, the entire code files as well as any source of language documentation can be used for teaching the language to the model. Instead, fine-tuning requires the building of natural language descriptions of the code to implement paired with a corresponding code implementation. In our setting (*i.e.*, function-level), this means mining from the very few repositories available for the no-resource language only the functions having a non-empty description, excluding everything else. This is the reason behind the much larger amount of training tokens available in the pre-training datasets (28.3M for Gleam, 13.7M for MoonBit) as compared to the fine-tuning datasets (3.6M for Gleam, 0.5M for MoonBit).

Similarly to what done in RQ₁, we looked at the impact of in-context learning, pre-training, and fine-tuning on the reduction of *syntactic* and *semantic* errors (full table in our replication package [26]). Within in-context learning, few-shot prompting is consistently more effective than RAG at reducing syntactic errors. This result holds across both languages and all four LLMs evaluated. For Gleam, even with few-shot learning syntactic errors remain the predominant cause of failure (over semantic ones) across all LLMs. In contrast, this pattern does not hold for MoonBit. Specifically, on the two Qwen models, $\sim 50\%$ of the failures are due to syntactic errors, while for the GPT-based models semantic errors represent roughly two-thirds of the overall failures. This again suggests a higher effectiveness of few-shot learning on MoonBit for the reasons previously explained, with LLMs getting a better understanding of the language syntax.

As expected, fine-tuning and pre-training lead to a substantial reduction in syntactic errors. Both approaches shift LLM behavior toward what we observed for high- and low-resource languages, with fewer than 20% of failures attributable to syntactic mistakes across all LLM-language combinations. These results confirm the superior effectiveness of training-based approaches compared to in-context learning methods for teaching no-resource languages to LLMs.

Answer to RQ₂

In-context learning techniques help in improving code generation performance for no-resource languages as compared to zero-shot. However, training-based techniques (fine-tuning and pre-training) are more effective than in-context learning, allowing open models to achieve performance superior to commercial LLMs. Pre-training is the most promising technique since, differently from fine-tuning, allows to exploit “all” little data available for the no-resource language, achieving the strongest boost in performance among all experimented techniques. On the negative side, the pre-trained base models do not have instruction-following capabilities, being thus suboptimal as AI coding assistants. In Section IV we address this limitation.

IV. INSTRUCTION TRANSFERRING

In our answer to RQ₂, we showed that the best approach to boost performance on no-resource languages is further pre-training a base model on the available data, even if scarce. However, the resulting model does not have instruction-following capabilities, which are crucial for an AI coding assistant. Indeed, prompts (*i.e.*, natural language descriptions of the desired code) can vary widely in form. This variability is already evident in existing benchmarks, which often use diverse prompting styles, but it becomes even more pronounced in real-world scenarios, where different developers may express the same request in very different ways.

To address this limitation, one possible solution would be to perform an additional *instruction fine-tuning* on top of the further pre-trained base model. However, (i) instruction-tuning datasets are typically not publicly available, and (ii) the cost of such a process is known to be extremely high [30], since large datasets are needed.

As an alternative approach, researchers in the Natural Language Processing (NLP) community recently suggested *fine-tuning reuse* [18], [19], which allows to transfer the instruction-following capabilities of a model M_i to another *base* model (*i.e.*, without instruction-following capabilities). In particular, such an application assumes the existence of three models all having the same size and architecture: (i) M_i , the one with instruction-following capabilities; (ii) M_b , the base model on top of which M_i has been created via instruction fine-tuning; and (iii) M_{bk} , a version of M_b further trained to better support a specific task or language of interest. By computing the diff between M_i 's and M_b 's weights (Δ_w), we can capture “the portion of M_i 's knowledge” allowing it to follow complex instructions. We can then sum Δ_w to M_{bk} 's weights, obtaining—at a negligible cost—a new instruct model (M_{bk+i}), which is specialized on the task/language of interest.⁴

We experiment this approach as a further attempt to boost the performance of code models on no-resource languages. In particular, we answer the following research question:

RQ₃: *To what extent does instruction transferring boost the code generation performance of LLMs on no-resource*

⁴Cost is negligible since the diff between models can be computed using CPUs only.

languages? Our M_{bk} are base models further pre-trained on the no-resource language of interest (as done in RQ₂), while M_i and M_b are the instruct and base versions of that same model, as released by their authors.

We use as M_i models the already mentioned Qwen 2.5 Coder 32B Instruct and Qwen 3 8B Instruct. The latter is a reasoning model, thus the weighting diff in this case is expected to inject reasoning capabilities into the M_{bk} models. For both models we have their M_b versions available (*i.e.*, Qwen 2.5 Coder 32B Base and Qwen 3 8B Base), thus allowing the computation of the weights diff Δ_w . Finally, our M_{bk} models are the versions of Qwen 2.5 Coder 32B Base and Qwen 3 8B Base further pre-trained on Gleam and MoonBit.

TABLE VI
LLMs PERFORMANCE WHEN USING BASE + PRE-TRAINING (PT) AND
BASE + PRE-TRAINING + INSTRUCTION TRANSFERRING (DIFF).

Difference between pre-training and instruction transferring						
Qwen 2.5 Coder 32B						
	Gleam			MoonBit		
Benchmark	PT	Diff	Δ	PT	Diff	Δ
HumanEval	32.99	56.23	▲ 23.24	41.62	50.71	▲ 9.09
MBPP	47.35	53.83	▲ 6.48	44.76	53.04	▲ 8.28
McEval Hard	12.47	26.08	▲ 13.61	25.86	32.60	▲ 6.74
Qwen 3 8B						
	Gleam			MoonBit		
Benchmark	PT	Diff	Δ	PT	Diff	Δ
HumanEval	18.57	51.88	▲ 33.31	36.82	44.42	▲ 7.60
MBPP	23.63	50.48	▲ 26.85	42.08	45.27	▲ 3.19
McEval Hard	4.36	22.33	▲ 17.97	19.87	19.82	▼ 0.05

Table VI reports the results achieved on the no-resource languages via instruction transferring (see column “Diff”), and their comparison against the best-performing approach highlighted in RQ₂, *i.e.*, the base model further pre-trained on the no-resource language (see column “PT”). In what follows, we discuss the differences between the two approaches, while also highlighting the improvement achieved by instruction transferring with respect to other techniques experimented in RQ₂ (Table V). Also in this case, we provide the full results of *passed%* in the replication package [26], highlighting relevant differences between the two metrics (*i.e.*, *pass@1* and *passed%*) in the following.

Instruction transferring yields a significant improvement over the base model further pre-trained on the no-resource language, with *pass@1* scores increasing by up to 33% (Gleam, HumanEval, Qwen 3), and an average increase across benchmarks, models and languages of 12%. All improvements are statistically significant (adjusted *p*-value < 0.05, McNemar test), with an OR ranging between 1.21 and 10.95. There is only one case where the instruction transferring approach does not improve performance, namely ⟨MoonBit, McEval-Hard, Qwen 3⟩, although the difference is small (0.05%) and not statistically significant (OR=1). Overall, we can safely state that instruction transferring significantly boosts the code generation capabilities of LLMs on no-resource languages. This boost is observed across different languages, benchmarks, and models. More importantly, it generalizes to models having different sizes (8B vs 32B), being general-purpose (Qwen 3) or specialized on code (Qwen 2.5 Coder), and with or without reasoning capabilities (Qwen 3 vs Qwen 2.5 Coder).

It is worth noting that instruction transferring provides a more substantial improvement on Gleam than on MoonBit. Our hypothesis is that this is due to the fact that the improvement achieved by the further pre-training on MoonBit was already quite high, with an average increase in the *pass@1* of 28% with respect to the base model (see Table V, Base, 0-shot). In contrast, the further pre-training on Gleam yielded a lower improvement (23%), leaving more room for the instruction transferring to boost performance.

When comparing the results against all LLMs and techniques evaluated in RQ₂ (Table V), we observe that instruction transferring is the best approach in all cases. For instance, for the Gleam language, besides the “further pre-training” strategy shown in Table VI, another high-performing approach is the fine-tuning of Qwen 3 32B Instruct, which achieved *pass@1* scores of 24% (HumanEval), 37% (MBPP), and 4% (McEval-Hard). Even so, the instruction transferring strategy applied on Qwen 2.5 Coder 32B Base achieves +33%, +17%, and +22% improvements over these scores (measured in absolute terms), respectively. The same holds for MoonBit, where the second best-performing approach (after the further pre-training) is o3-mini complemented with a 5-shot in-context learning, which reached *pass@1* scores of 39% (HumanEval), 46% (MBPP), and 12% (McEval-Hard). Again, the instruction transferring approach applied on Qwen 2.5 Coder 32B Base achieves +11%, +7%, and +20% improvements over these scores, respectively.

Lastly, we draw attention to an important finding from our experiments: instruction transferring applied to smaller models can result in outperforming larger and more expensive models. Indeed, Qwen 3 8B Base complemented with instruction transferring consistently outperforms Qwen 3 32B Instruct, whatever technique is applied on it.

As compared to a fine-tuned Qwen 3 32B (best baseline from Table V), Qwen 3 8B with instruction transferring obtained up to +28% (measured in absolute terms) increments in *pass@1* scores over the fine-tuned version of Qwen 3 32B Instruct (Gleam, HumanEval), with an average increase of 12% across all benchmarks and languages. This finding highlights the exceptional efficiency of the instruction transferring technique, which allows smaller models to achieve superior performance compared to larger (4×) alternatives. This is particularly relevant for practitioners working with computational constraints, as it suggests that strategic knowledge transfer can be more valuable than simply scaling model size.

While the results of the *passed%* metric are consistent with what we discussed in terms of *pass@1*, we noticed one single difference worth being discussed.

Transitioning from pre-training to instruction transferring substantially reduces the total number of syntactic errors for the large model (32B): Qwen 2.5 exhibits a reduction of 24.1% in syntactic errors on Gleam (from 752 to 571) and 73.5% on MoonBit (from 645 to 171). Instead, on the small model (8B), besides an overall improvement in terms of performance, instruction transferring caused the failures to shift more towards syntactical errors as compared to the pre-trained model, both on Gleam (+13.3%, from 460 to 521) and on MoonBit (+241.6%, from 545 to 1,863). Clearly, such an

increase of syntactical errors is accompanied by a stronger decrease of semantic errors, justifying the overall boost in performance. This divergent behavior between model sizes can be explained by differences in model capacity. For the larger model, instruction transferring effectively reinforces both syntactic competence and task-level understanding, leading to a substantial reduction of both syntactic and semantic errors. In contrast, the smaller model appears to reallocate its limited capacity toward improved instruction following and semantic reasoning. As a result, while overall performance improves, the remaining failures are more frequently attributable to syntactic issues. This shift also explains the only discrepancy we observed between the indications provided by $pass@1$ and those provided by $passed\%$. Specifically, for Qwen 8B on the MoonBit translation of HumanEval, instruction transferring yields a higher $pass@1$ than pre-training (44.42 vs 36.82), but a slightly lower $passed\%$ (46.58 vs 47.95). A plausible explanation is that syntactic errors, more frequent with instruction transferring, often prevent execution altogether, causing all tests for a task to fail rather than only a subset. Consequently, a model with more syntactic errors may still achieve a higher $pass@1$ by fully solving more tasks, while obtaining a lower $passed\%$ because, on the tasks it fails, it more frequently fails all tests instead of only some of them.

Answer to RQ₃

Instruction transferring provides a strong boost in performance on no-resource languages as compared to the best-performing technique we experimented with (*i.e.*, further pre-training). Such a boost comes almost for free, and allows even smaller models to become competitive with larger ones.

V. VALIDITY DISCUSSION

A. Experimental Procedure and Evaluated Techniques

We did not perform hyperparameter tuning of the experimented LLMs as this would have required a significant amount of computational resources. We used the default configurations suggested by the authors of the models. As for the number of training epochs (5), this was dictated by the monitoring of the loss function over training. As a double-check, we tested the models also after the first and third epochs, always getting worst results for the first epoch (indicating the need for more training) and quite similar results for the third (*i.e.*, at most a $\pm 3.4\%$ gap in $pass@1$ when considering all combinations of LLMs and languages).

For all experimented techniques we had to make choices. For in-context learning techniques, all prompts and scripts we used are publicly available [26]. Both in-context learning techniques we experiment with (*i.e.*, few-shot prompting and RAG) dynamically select the most relevant code examples and documentation to be included in the prompt. However, this additional context may not always capture all language features required by the LLM to solve the task at hand. To assess the extent to which this could have impacted our findings, we implemented a third prompting strategy, consisting in a language-specific manual covering all information necessary to solve the tasks in our three benchmarks. To construct this

manual, we first queried the documentation to identify the files most relevant to each benchmark task. This enabled us to reduce the full documentation to a representative subset of files relevant to at least one task. We then manually inspected the retrieved files to confirm their relevance to the corresponding benchmark tasks. Finally, we prompted GPT-4.1 to summarize these documents into a concise manual including short explanations and code examples for each required programming concept (the summarization prompt and the resulting manuals are available in our replication package [26]). The first two authors manually verified the accuracy of the generated Gleam and MoonBit manuals against the original documentation. The resulting manual can then be used as fixed prompt context for each code generation request submitted to an LLM.

We evaluated this strategy on Qwen 2.5 Coder 32B Instruct, as it is the only model for which we report results across all experimental settings (in-context learning, fine-tuning, continuous pre-training, and instruction transferring). As in the previous experiments, we performed 10 runs to account for the stochastic nature of LLM outputs. Before discussing the results, we note that this methodology, as applied here, is mainly useful for identifying an approximate upper bound on the performance achievable with in-context learning techniques. Indeed, the manual is effectively “overfitted” to a set of known code-generation tasks, namely those included in the benchmarks, which would not be known *a priori* in a realistic usage scenario.

We obtained a $pass@1$ of 15.58% on HumanEval, 25.07% on MBPP, and 1.76% of McEval-Hard for Gleam, and 32.47% on HumanEval, 39.44% on MBPP, and 6.61% of McEval-Hard for MoonBit. While results on the first two benchmarks are significantly better than any other in-context learning technique, and nearly on par with the fine-tuning baseline, they are still underperforming against our best technique (*i.e.*, instruction transferring). This is especially evident in the most challenging benchmark (*i.e.*, McEval-Hard), where it achieved a lower $pass@1$ (Gleam 1.76%, MoonBit 6.61%) compared to the model after instruction transferring (Gleam 26.08%, MoonBit 32.60%).

Finally, in RQ₂ we observed that LLMs mainly fail on no-resource languages due to syntactical errors. One possible approach to mitigate these errors is via constrained decoding techniques [36], [37], which enforce language grammar rules at inference time. While we acknowledge that these techniques can boost open-weight models’ performance without additional training, they do not take into account language APIs, semantic correctness, and language-specific coding conventions. Therefore, we did not include these techniques in our study but we plan to experiment with them in future work.

B. Experimental Assumptions

A key assumption underlying our study is that the evaluated LLMs have not been exposed to Gleam or MoonBit code during training. However, the training corpora of the evaluated LLMs are not publicly characterized at a level that would allow us to determine whether Gleam or MoonBit code was present. For the open models considered in this work, we therefore

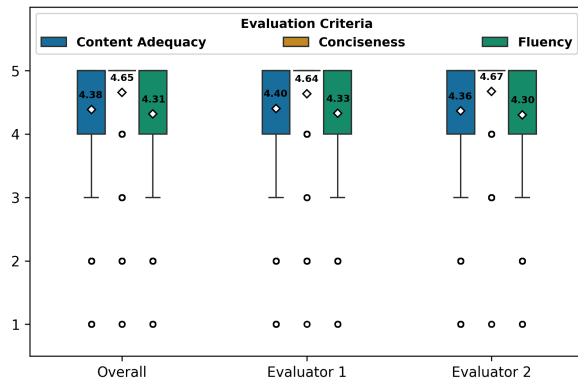


Fig. 1. Distribution of the quality scores for the three evaluated criteria on the fine-tuning datasets.

cannot verify the absence of these languages from the training data. We found that neither Gleam nor MoonBit appeared among the 92 supported languages listed in the `README.md` file associated with the commit introducing Qwen 2.5 Coder [38]. Similarly, the most recent version of `README.md` for Qwen 3 [39] omits Gleam and MoonBit from the 358 supported programming languages. We interpret this only as evidence that these languages are not advertised as supported by the models, rather than as evidence about the contents of the training corpora. Nevertheless, the languages' timelines suggest that any such exposure was likely limited: Gleam reached version 1.0 only in March 2024, while MoonBit was still undergoing alpha testing in late 2023 and reached beta in June 2025.

C. Data Quality

One of the techniques we explore to improve LLM performance on no-resource languages is fine-tuning. This approach relies on the automated collection of $\langle doc, function \rangle$ pairs, where *doc* denotes a natural-language description of the code to be implemented (provided as input to the LLM), and *function* represents the corresponding target code generation. In this process, the quality of the collected data is critical to the effectiveness of fine-tuning. To assess the quality of the fine-tuning datasets, we manually validated a random sample of 374 instances from the Gleam dataset and 333 instances from the MoonBit dataset. Both samples are statistically significant, ensuring a 95% confidence level with a $\pm 5\%$ margin of error within each corresponding fine-tuning dataset. The first two authors manually validated the quality of the 707 summaries (*i.e.*, the *doc* associated to each function) by following the same procedure used in works assessing the quality of manually [40] or automatically-generated [41] summaries. In particular, each summary has been independently assessed by the two evaluators across three dimensions: *content adequacy*, *conciseness*, and *fluency & understandability*. Each quality attribute has been assessed on a scale from 1 to 5 (the higher the better), using the guidelines defined by Crupi *et al.* [40].

We computed the agreement among the human judges for the three quality aspects using the Krippendorff α [42],

obtaining 0.69 for *content adequacy* (substantial agreement), 0.48 for *conciseness* (moderate), and 0.65 for *fluency & understandability* (substantial). Fig. 1 shows the distribution of the three quality attributes when considering the evaluations provided by: (i) both evaluators as a single distribution—overall; (ii) evaluator 1; and (iii) evaluator 2. As it can be seen, the median both overall as well as for each individual evaluator, is always 5, with average values never going below 4.3. These results give us confidence about the overall quality of the fine-tuning datasets we experimented with. Still, as for any dataset collected in the wild, we acknowledge the presence of low-quality instances which, however, represent the minority of the inspected instances (see Fig. 1). We also acknowledge a possible validity threat because the analysis was performed post-hoc by the first two authors, which may have introduced some bias in the evaluation. We tried to minimize such a bias by following the evaluation procedure described above.

Also, our findings could be influenced by errors made during the benchmarks' translations. To check the quality of the translated benchmarks, we randomly selected a set of instances for a further manual inspection. In particular, we created a statistically-significant sample ($95\% \pm 5\%$) of translated coding tasks, stratified across the nine languages involved in our study. Our population is composed by a total of 3,061 translated coding tasks (*i.e.*, $227 \times 9 = 2,043$ for the creation of McEval-Hard, $154 \times 2 = 308$ for the translation of HumanEval in Gleam and MoonBit, and $355 \times 2 = 710$ for the translation of MBPP in Gleam and MoonBit). Given such a population, the selected sample features 346 coding tasks, with 82 being Gleam, 82 being MoonBit, and the remaining 182 equally split across the remaining 7 languages (26 each). Basically, in such a sample we include more instances for more represented languages in the translated benchmarks (*i.e.*, Gleam and MoonBit). Then, the first two authors independently looked at each instance, comparing it with the original coding task from which it has been translated. Both authors had to answer two boolean questions: (i) is the prompt (*i.e.*, code description) of the translated coding task equivalent to that of the original task it stems from?; and (ii) are the tests of the translated coding task equivalent to those of the original task it stems from? We also used this analysis to check for possible errors inherited by the source benchmarks (*e.g.*, misalignments between what required in the `docstring` and what tested in the test suite). There were 23 cases (6.6%) in which the evaluators disagreed in the equivalence/correctness of the prompt and 14 (4%) of the tests. We solved conflicts via open discussion. As a result, we found 14 prompts (4%) and two tests (0.6%) presenting issues. The two instances with incorrect test translations originate from task CPP/17 in McEval-Hard. Specifically, the original test suite validates the function robustness by passing binary and hexadecimal literals as arguments. In our R and Lua translations, however, we converted these numbers to decimals (*e.g.*, translating `0b1101110111` to `1775`). Although these translations are partially accurate (*e.g.*, Lua lacks native binary literals), they may not be considered as fully equivalent to the original tests. Regarding the 14 prompts we flagged as prob-

lematic: five errors were inherited issues from the McEval-Hard benchmarks that we did not spot while translating the original benchmark, such as doctests with incorrect return values; four were typos/textual deviations from the original prompt, such as missing a sentence; three related to the lack of type-related information, such as missing type hints in the function arguments; two were instead type-related errors (*e.g.*, in MoonBit one doctest reported “true” as the expected return value, while it should be “True”). Overall, we detected a very low percentage of potential quality issues, which are inherent of any built benchmark, as also shown by recent work investigating the quality of code generation benchmarks [43], [44]. For example, Wang *et al.* [44] recently showed that test suites in SWE-bench have significant flaws, leading to inflated issue-resolution rates by 6.4%, on average.

D. Generalizability

We experimented with nine languages, six LLMs, and three benchmarks, our findings may not generalize to other settings. Also, the specific tasks we used to test the selected LLMs may not reflect the real usage scenarios in which programming languages are used. Indeed, some programming languages might be more frequently used in specific domains, such as high-performance computing, game development, or IoT. While this is not the case for MoonBit, which is described as a general-purpose language in their official website [45], Gleam is mainly optimized for distributed systems and web development. To complement our analysis, future work should target domain-specific tasks for these programming languages.

Finally, our work does not aim at improving LLMs for Gleam and MoonBit specifically, but rather at studying the gap between no-resource languages and higher-resource ones, as well as proposing and evaluating techniques to improve LLMs in no-resource settings. In this sense, the implications of our study remain relevant even if future LLMs are trained on Gleam and MoonBit.

VI. RELATED WORK

In recent years, numerous code generation techniques [1], [3], [46], [47] and benchmarks [11], [12], [48], [49] have been proposed, with most of them targeting popular programming languages, such as Python and Java. In this section, we focus on code generation benchmarks presented for low-resource programming languages and past attempts aiming at improving LLMs’ performance in this scenario.

Code Generation Benchmarks for Low-Resource Languages. Several works proposed the translation of monolingual benchmarks to multiple programming languages, including low-resource ones [5], [50], [51]. A notable example is MULTIPL-E [5], in which the authors translated HUMAN-EVAL [11] and MBPP [12] to 24 languages. We exploit the MULTIPL-E translated benchmarks to assess the performance of several LLMs on the low-resource programming languages subject of our study. Also, we translated those same benchmarks in Gleam and MoonBit (no-resource languages).

Due to the raising concerns of data contamination between LLMs’ training corpora and popular benchmarks, Chai *et al.*

released MCEVAL [13], a benchmark featuring 2k human-written code generation tasks. MCEVAL supports 40 programming languages, with about 50 code instances each. We revised the original McEval benchmark to create a more challenging and robust version of it for low- and no-resource programming languages.

Empirical Studies on Low-Resource Programming Languages. There are several works tackling the code generation problem for low-resource languages, some of which have been explicitly crafted for a given language, hardly being generalizable to other languages. For example, Kogler *et al.* [52] proposed the use of an intermediate DSL crafted as a JSON for a specific task and language of interest (*e.g.*, generating test specifications using the Balise Telegram Test Language). As also highlighted by the authors, their technique is not easily generalizable to other scenarios. In this section, we discuss techniques which, as the ones we experimented with, can be applied to any language. For a more complete overview of the area which includes the more specialized techniques, we invite the reader to check the recent survey by Joel *et al.* [53].

Early attempts to improve performance on low-resource languages [35], [51], [54] deal with the shortage of training data by fine-tuning on multilingual datasets. For example, Chen *et al.* [35] found that training a DL model on programming languages similar to the niche ones of interest can boost performance on the latter. Our findings confirm, at least in part, what observed by Chen *et al.* Indeed, the LLMs used in our study have been pre-trained on a multitude of programming languages and worked well on some of the low-resource languages we experimented with. Instead, the multilingual pre-training did not help on the no-resource languages.

Cassano *et al.* [4] proposed MULTIPL-T, a framework to generate synthetic training data for low-resource languages. They used MULTIPL-T to translate Python functions to multiple low-resource languages, using LLMs and language-specific compilers. They found that this technique can improve LLMs code generation performance on five low-resource languages. Paul *et al.* [55] leveraged LLVM’s intermediate representation [56] to align code from popular and low-resource languages through a shared representation. They found that by continuously pre-training LLMs on pairs of source code and its intermediate representation, the code generation abilities of these models are greatly enhanced on niche languages.

In our study, we did not include MULTIPL-T and the approach by Paul *et al.* [55] as baselines since the former cannot be applied in the context of no-resource languages, given the inability of LLMs to support the automated translation from high- to no-resource languages. For the latter, instead, there is no support of the LLVM Compiler Infrastructure for Gleam and MoonBit.

Other studies explored in-context learning techniques in the low-resource scenario. Athiwaratkun *et al.* [50] investigated few-shot learning as an alternative method for teaching new languages to code models. Their experiments show that prepending some code examples on the original prompt can help a model to generate more accurate code. Dutta *et al.* [57] proposed RAR, a retrieval-augmented technique to guide

model completion in low-resource languages. RAR uses a two-step retrieval process that relies on language documentation: it first retrieves relevant grammar definitions (*i.e.*, classes, methods, properties) given the code completion context and then extracts code examples from these. Their experiments show great improvements over existing baselines. Finally, Giagnorio *et al.* [6] analyzed several in-context learning and fine-tuning strategies to improve models' code abilities in low-resource programming languages. Their findings reveal that providing more examples in the LLM context aids the model to generate better code, while fine-tuning techniques may actually reduce model performance. We used few-shot, RAG, and fine-tuning as baselines.

In a related line of research, Costa *et al.* recently proposed ModelMate [58], an approach that fine-tunes pre-trained language models to provide editor assistance for textual domain-specific languages (DSLs) with little or no available training data. Their key idea is to increase the amount of training data by converting models written in a semantically compatible modeling language into the target DSL through a model-to-text transformation. This approach, however, relies on the assumption that such a transformation can be constructed for the target language. While this assumption is reasonable for DSLs with narrow, domain-constrained semantics, our work instead focuses on no-resource general-purpose programming languages, which exhibit broader syntax and semantics and are used for end-to-end program synthesis, making such assumptions less applicable.

VII. CONCLUSION AND FUTURE WORK

We empirically evaluated the code generation capabilities of six state-of-the-art LLMs on high-, low-, and no-resource programming languages (*i.e.*, languages having abundant, little, and almost no training data, respectively). To run such an evaluation, we invested major effort in the creation of (i) benchmarks for two no-resource languages (Gleam and MoonBit), and (ii) a more complex benchmark for the experimented high- and low-resource languages (*i.e.*, McEval-Hard). This required a total of ~ 340 man hours. All our benchmarks are available to foster research in code generation [26].

On McEval-Hard, LLMs achieved a *pass@1* in the range of ~ 59 - 89% for high-resource languages, 27 - 84% for low-resource, and 0 - 1% for no-resource. We then investigated possible solutions to boost performance on no-resource languages, with the *pre-training* of base models on the little data available resulting as the most effective technique, achieving a *pass@1* up to 12% on McEval-Hard for Gleam and 26% for MoonBit. We also explored *fine-tuning reuse* to transfer the instruction-tuned weights of an instruct model to a base model specialized (via pre-training) on the no-resource language. This approach considerably outperformed the other techniques, achieving a *pass@1* of up to 26% for Gleam and 33% for MoonBit on McEval-Hard. If we compare these values to what originally observed in a zero-shot setting for the same languages (*i.e.*, *pass@1* close to 0), our empirical investigation represents a starting point for companies interested in training and deploying their in-house AI-based coding assistant specialized on proprietary languages.

Our future work will target the expansion of the current benchmarks to cover a more diverse set of real-world use cases in no-resource languages, like debugging, refactoring, or the generation of code aimed at addressing entire change requests.

ACKNOWLEDGMENTS

Giagnorio and Bavota acknowledge the financial support of the Swiss National Science Foundation for the PARSED project (SNF Project No. 219294). We also thank Louis Pilfold, creator of the Gleam language, for the help in verifying the correctness of Gleam benchmarks, and the entire Gleam and MoonBit communities for their support in the development of the benchmarks.

REFERENCES

- [1] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: language agents with verbal reinforcement learning," in *Proceedings of the 37th International Conference on Neural Information Processing Systems*, 2023, pp. 8634–8652.
- [2] J. Cao, Z. Chen, J. Wu, S.-C. Cheung, and C. Xu, "Javabench: A benchmark of object-oriented code generation for evaluating large language models," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024, pp. 870–882.
- [3] M. A. Islam, M. E. Ali, and M. R. Parvez, "Mapcoder: Multi-agent code generation for competitive problem solving," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 4912–4944.
- [4] F. Cassano, J. Gouwar, F. Lucchetti, C. Schlesinger, A. Freeman, C. J. Anderson, M. Q. Feldman, M. Greenberg, A. Jangda, and A. Guha, "Knowledge transfer from high-resource to low-resource programming languages for code llms," *Proceedings of the ACM on Programming Languages*, vol. 8, no. OOPSLA2, pp. 677–708, 2024.
- [5] F. Cassano, J. Gouwar, D. Nguyen, S. Nguyen, L. Phipps-Costin, D. Pinckney, M.-H. Yee, Y. Zi, C. J. Anderson, M. Q. Feldman *et al.*, "Multipl-e: a scalable and polyglot approach to benchmarking neural code generation," *IEEE Transactions on Software Engineering*, 2023.
- [6] A. Giagnorio, A. Martin-Lopez, and G. Bavota, "Enhancing code generation for low-resource languages: No silver bullet," in *33rd IEEE/ACM International Conference on Program Comprehension, ICPC/ICSE 2025, Ottawa, ON, Canada, April 27-28, 2025*, 2025, pp. 478–488.
- [7] "GitHub Copilot – Your AI pair programmer," <https://github.com/features/copilot/>, accessed: 2024-03-10.
- [8] "Chatgpt," <https://openai.com/blog/chatgpt>.
- [9] "Gleam language," <https://gleam.run/>, [n.d.], accessed: 2025-07-01.
- [10] H. Fei, Y. Zhang, H. Zhang, Y. Wang, and Q. Liu, "Moonbit: Explore the design of an ai-friendly programming language," in *Proceedings of the 1st International Workshop on Large Language Models for Code*, 2024, pp. 79–83.
- [11] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.
- [12] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le *et al.*, "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021.
- [13] L. Chai, S. Liu, J. Yang, Y. Yin, J. Liu, T. Sun, G. Zhang, C. Ren, H. Guo, N. Wang *et al.*, "Mceval: Massively multilingual code evaluation," in *13th International Conference on Learning Representations*, 2025.
- [14] "Gpt-4o," <https://platform.openai.com/docs/models/gpt-4o>.
- [15] "o3-mini," <https://platform.openai.com/docs/models/o3-mini>.
- [16] "Qwen2.5 coder 32b instruct," <https://huggingface.co/Qwen/Qwen2.5-Coder-32B-Instruct>.
- [17] "Qwen3 32b," <https://huggingface.co/Qwen/Qwen3-32B>.
- [18] I. Jindal, C. Badrinath, P. Bharti, L. Vinay, and S. D. Sharma, "Balancing continuous pre-training and instruction fine-tuning: Optimizing instruction-following in llms," *arXiv preprint arXiv:2410.10739*, 2024.
- [19] P.-J. Lin, R. Balasubramanian, F. Liu, N. Kandpal, and T. Vu, "Efficient model development through fine-tuning transfer," *arXiv preprint arXiv:2503.20110*, 2025.

- [20] T. Van Dam, F. Van der Heijden, P. De Bekker, B. Nieuwschepen, M. Otten, and M. Izadi, "Investigating the performance of language models for completing code in functional programming languages: A Haskell case study," in *2024 IEEE/ACM First International Conference on AI Foundation Models and Software Engineering (Forge)*. IEEE Computer Society, 2024, pp. 91–102.
- [21] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. R. Narasimhan, "SWE-bench: Can language models resolve real-world github issues?" in *The Twelfth International Conference on Learning Representations*, 2024.
- [22] "Qwen2.5 coder 32b," <https://huggingface.co/Qwen/Qwen2.5-Coder-32B>.
- [23] "Qwen3 8b base," <https://huggingface.co/Qwen/Qwen3-8B-Base>.
- [24] N. Muennighoff, Z. Yang, W. Shi, X. L. Li, L. Fei-Fei, H. Hajishirzi, L. Zettlemoyer, P. Liang, E. Candès, and T. Hashimoto, "s1: Simple test-time scaling," *arXiv preprint arXiv:2501.19393*, 2025.
- [25] H. Face, "Open r1: A fully open reproduction of deepseek-r1," January 2025. [Online]. Available: <https://github.com/huggingface/open-r1>
- [26] "Replication package," <https://doi.org/10.5281/zenodo.19366887>, [n.d.].
- [27] "Openai text embedding," <https://platform.openai.com/docs/models/text-embedding-3-large>.
- [28] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazaré, M. Lomeli, L. Hosseini, and H. Jégou, "The faiss library," *arXiv preprint arXiv:2401.08281*, 2024.
- [29] "gpt-4o-mini," <https://platform.openai.com/docs/models/gpt-4o-mini>.
- [30] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations*, 2022.
- [31] M. Weyssow, X. Zhou, K. Kim, D. Lo, and H. Sahraoui, "Exploring parameter-efficient fine-tuning techniques for code generation with large language models," *ACM Transactions on Software Engineering and Methodology*, 2023.
- [32] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *7th International Conference on Learning Representations, ICLR*, 2019.
- [33] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.
- [34] B. Yoav and H. Yosef, "Controlling the false discovery rate: A practical and powerful approach to multiple testing," *Journal of the Royal Statistical Society. Series B (Methodological)*, vol. 57, no. 1, pp. 289–300, 1995.
- [35] F. Chen, F. Fard, D. Lo, and T. Bryksin, "On the transferability of pre-trained language models for low-resource programming languages," in *30th IEEE/ACM International Conference on Program Comprehension, ICPC*, 2022, pp. 401–412.
- [36] B. T. Willard and R. Louf, "Efficient guided generation for large language models," *arXiv preprint arXiv:2307.09702*, 2023.
- [37] Y. Dong, C. F. Ruan, Y. Cai, Z. Xu, Y. Zhao, R. Lai, and T. Chen, "Xgrammar: Flexible and efficient structured generation engine for large language models," *Proceedings of Machine Learning and Systems*, vol. 7, 2025.
- [38] "Qwen 2.5 coder: Supported languages," <https://github.com/QwenLM/Qwen3-Coder/blob/7a7faf8449e2b94897a4d9d4e287f80b99c17f1/README.md>.
- [39] "Qwen 3: Supported languages," <https://github.com/QwenLM/Qwen3-Coder?tab=readme-ov-file#basic-information>.
- [40] G. Crupi, R. Tufano, A. Velasco, A. Mastropaolo, D. Poshyvanyk, and G. Bavota, "On the effectiveness of llm-as-a-judge for code generation and summarization," *IEEE Trans. Software Eng.*, vol. 51, no. 8, pp. 2329–2345, 2025.
- [41] D. Roy, S. Fakhoury, and V. Arnaoudova, "Reassessing automatic evaluation metrics for code summarization tasks," in *29th ACM Joint Meeting on European Software Engineering Conference and the ACM/SIGSOFT Symposium on the Foundations of Software Engineering, ESEC-FSE*, 2021, p. 1105–1116.
- [42] K. Krippendorff, "Reliability in content analysis: Some common misconceptions and recommendations," *Human communication research*, vol. 30, no. 3, pp. 411–433, 2004.
- [43] M. L. Siddiq, S. Dristi, J. Saha, and J. C. S. Santos, "The fault in our stars: Quality assessment of code generation benchmarks," in *2024 IEEE International Conference on Source Code Analysis and Manipulation (SCAM)*, 2024, pp. 201–212.
- [44] Y. Wang, M. Pradel, and Z. Liu, "Are 'solved issues' in swe-bench really solved correctly? an empirical study," in *48th International Conference on Software Engineering, ICSE '26*. ACM, 2026, p. To appear.
- [45] "Moonbit website," <https://www.moonbitlang.com/>, [n.d.], accessed: 2025-07-01.
- [46] R. Li, L. Allal, Y. Zi, N. Muennighoff, D. Kocetkov, C. Mou, M. Marone, C. Akiki, J. Li, J. Chim *et al.*, "StarCoder: May the source be with you!" *Transactions on machine learning research*, 2023.
- [47] Z. Luo, C. Xu, P. Zhao, Q. Sun, X. Geng, W. Hu, C. Tao, J. Ma, Q. Lin, and D. Jiang, "WizardCoder: Empowering code large language models with evol-instruct," in *The Twelfth International Conference on Learning Representations*, 2024.
- [48] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation," *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [49] T. Y. Zhuo, M. C. Vu, J. Chim, H. Hu, W. Yu, R. Widyasari, I. N. B. Yusuf, H. Zhan, J. He, I. Paul, S. Brunner, C. Gong, J. Hoang, A. R. Zebaze, X. Hong, W. Li, J. Kaddour, M. Xu, Z. Zhang, P. Yadav, and *et al.*, "Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions," in *13th International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*, 2025.
- [50] B. Athiwaratkun, S. K. Gouda, Z. Wang, X. Li, Y. Tian, M. Tan, W. U. Ahmad, S. Wang, Q. Sun, M. Shang, S. K. Gonugondla, H. Ding, V. Kumar, N. Fulton, A. Farahani, S. Jain, R. Giaquinto, H. Qian, M. K. Ramanathan, and R. Nallapati, "Multi-lingual evaluation of code generation models," in *11th International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*, 2023.
- [51] G. Orlanski, K. Xiao, X. Garcia, J. Hui, J. Howland, J. Malmaud, J. Austin, R. Singh, and M. Catasta, "Measuring the impact of programming language distribution," in *International Conference on Machine Learning*. PMLR, 2023, pp. 26619–26645.
- [52] P. Kogler, W. Chen, and S. Wallner, "Code generation for niche programming languages with large language models," in *Software Engineering 2025—Companion Proceedings*. Gesellschaft für Informatik, Bonn, 2025, pp. 10–18420.
- [53] S. Joel, J. J. Wu, and F. H. Fard, "A survey on llm-based code generation for low-resource and domain-specific programming languages," *arXiv preprint arXiv:2410.03981*, 2024.
- [54] T. Ahmed and P. T. Devanbu, "Multilingual training for software engineering," in *44th IEEE/ACM International Conference on Software Engineering, ICSE*, 2022, pp. 1443–1455.
- [55] I. Paul, G. Glavaš, and I. Gurevych, "Ircoder: Intermediate representations make language models robust multilingual code generators," in *62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 15023–15041.
- [56] C. Lattner and V. Adve, "Llvm: A compilation framework for lifelong program analysis & transformation," in *International symposium on code generation and optimization, 2004. CGO 2004*. IEEE, 2004, pp. 75–86.
- [57] A. Dutta, M. Singh, G. Verbruggen, S. Gulwani, and V. Le, "Rar: Retrieval-augmented retrieval for code generation in low resource languages," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024, pp. 21506–21515.
- [58] C. D. Costa, J. A. H. López, and J. S. Cuadrado, "Modelmate: A recommender for textual modeling languages based on pre-trained language models," in *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems, ser. MODELS '24*, 2024, p. 183–194.